

ALLMod: Exploring Area-Efficiency of LUT-based Large Number Modular Reduction via Hybrid Workloads

Fangxin Liu^{1,2,†}, Haomin Li^{1,2,†}, Zongwu Wang^{1,2}, Bo Zhang³, Mingzhe Zhang³, Shoumeng Yan³,
Li Jiang^{1,2,*}, and Haibing Guan¹

1. Shanghai Jiao Tong University, 2. Shanghai Qi Zhi Institute, 3. Ant Group

*Corresponding Author {liufangxin, haominli, ljiang_cs}@sjtu.edu.cn

Abstract—Modular arithmetic, particularly modular reduction, is widely used in cryptographic applications such as homomorphic encryption (HE) and zero-knowledge proofs (ZKP). High-bit-width operations are crucial for enhancing security; however, they are computationally intensive due to the large number of modular operations required. The lookup-table-based (LUT-based) approach, a “space-for-time” technique, reduces computational load by segmenting the input number into smaller bit groups, pre-computing modular reduction results for each segment, and storing these results in LUTs. While effective, this method incurs significant hardware overhead due to extensive LUT usage. In this paper, we introduce ALLMod, a novel approach that improves the area efficiency of LUT-based large-number modular reduction by employing hybrid workloads. Inspired by the iterative method, ALLMod splits the bit groups into two distinct workloads, achieving lower area costs without compromising throughput. We first develop a template to facilitate workload splitting and ensure balanced distribution. Then, we conduct design space exploration to evaluate the optimal timing for fusing workload results, enabling us to identify the most efficient design under specific constraints. Extensive evaluations show that ALLMod achieves up to 1.65× and 3× improvements in area efficiency over conventional LUT-based methods for bit-widths of 128 and 8,192, respectively.

I. INTRODUCTION

Privacy computing, a crucial approach for safeguarding data security, has gained considerable attention in recent years due to increasing concerns about privacy and the protection of personal data on the Internet. This field encompasses a variety of applications, including homomorphic encryption (HE) [1], which protects the privacy of user data and models, and zero-knowledge proofs (ZKP) [2], [3], which ensure user privacy during transactions. These applications rely on public-key cryptography algorithms, such as Elliptic Curve Cryptography (ECC) [4] and RSA [5], which use modular reduction as a fundamental component to prevent overflow.

The modular reduction used in these algorithms often requires extremely high bit-widths. For instance, ECC demands

[†] These authors contributed equally. This work was partially supported by the National Key Research and Development Program of China (2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), and Natural Science Foundation of Shanghai (Grant No.24ZR1433700). This work was also supported by Ant Group through CCF-Ant Research Fund (CCF-AFSG RF20240304).

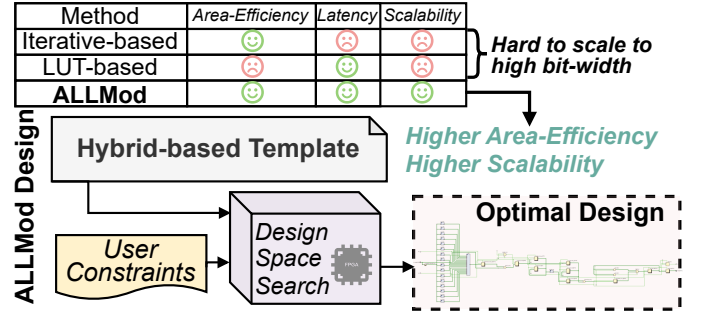


Fig. 1. Comparison of ALLMod to the existing modular reduction methods and ALLMod design overview.

a bit-width of at least 224 bits [6], while RSA typically requires at least 1,024 bits [7]. Moreover, for most cryptographic schemes, higher bit-widths correspond to enhanced security [8]. Such high-bit-width operations pose challenges for efficiently deploying these algorithms and their applications on hardware. Common algorithms, like Barrett [9] and Montgomery [10], implement modular operations through multiplication. However, as bit-width increases, constructing high-performance multipliers to meet latency and throughput requirements becomes increasingly difficult.

Recently, lookup-table-based (LUT-based) methods [11], [12] have been proposed to achieve low-latency and high-throughput large-number modular reduction. This approach divides input bits into segments and pre-computes possible modular reduction results for each segment. These results are organized into lookup tables (LUTs), which can be implemented using block RAMs (BRAMs) on FPGAs. For each segment, the corresponding modular result is retrieved from the LUTs and combined through an adder tree to yield the final modular reduction result.

This approach converts complex modular reduction operations into simple lookup operations within LUTs, significantly reducing operational complexity and enabling high-throughput, low-latency large-number modular reduction. As a “space-for-time” trade-off, it utilizes BRAMs to store the LUTs and DSPs to implement adder trees that aggregate lookup results. However, as bit-width increases, this method

faces the challenge of excessive resource overhead, resulting in suboptimal area efficiency. On the other hand, the iterative method, though straightforward, requires only a minimal number of subtractors but exhibits high latency.

To address these challenges, we propose combining iterative and LUT-based methods to enhance area efficiency in modular reduction implementations. Our solution, ALLMod, reduces area overhead by partitioning input bits into distinct workloads and distributing these across different modular modules, maintaining high throughput. To facilitate workload partition and achieve balanced workload distribution, we design a template that guides implementation. Building on this, we introduce a design space exploration method to identify the optimal design under specified constraints. Experimental results show that ALLMod improves area efficiency by up to $3\times$ compared to the conventional LUT-based method.

In summary, our contributions are as follows:

- We present a novel method to improve the area efficiency of LUT-based large-number modular reduction by smartly distributing the workload, enabling the integration of the iterative method with the LUT-based approach.
- We design a flexible template to facilitate workload division for implementations of varying bit-widths. This template, informed by latency analysis of both methods, ensures balanced workloads while minimizing area usage.
- Using such a template, we propose a design space exploration approach focused on achieving high area efficiency. This approach includes a resource evaluator and an iterative search algorithm, enabling the identification of Pareto-optimal designs under specified constraints, such as latency and area requirements.

II. BACKGROUND

A. Large-Number Modular Reduction Methods

In cryptographic algorithms, large-number modular reduction can be expressed as $R = A \bmod M$, where A is a $2n$ -bit number and M is a fixed n -bit number.

1) *LUT-based Method*: The LUT-based method [11], [12] employs lookup tables to store precomputed modular reduction results for possible segments of A , effectively reducing the computational load during runtime at the cost of additional space overhead. To leverage LUTs, the modular reduction operation must be transformed as follows:

$$\begin{aligned}
 R &= A \bmod M = \sum_{i=0}^{2n-1} (2^i a_i) \bmod M \\
 &= \left(\sum_{i=n}^{2n-1} (2^i a_i) + \sum_{i=0}^{n-1} (2^i a_i) \right) \bmod M \\
 &= \left(\left(\sum_{i=0}^{\frac{n}{k}-1} \sum_{j=0}^{k-1} (2^{n+ki+j} a_{n+ki+j}) \right) + \sum_{i=0}^{n-1} (2^i a_i) \right) \bmod M \\
 &= \left(\sum_{i=0}^{\frac{n}{k}-1} ((\hat{a}_i \times 2^{n+ki}) \bmod M) + \sum_{i=0}^{n-1} (2^i a_i) \right) \bmod M
 \end{aligned} \tag{1}$$

Algorithm 1: LUT-based Modular Reduction [12].

Data: $2n$ -bit number $A = \sum_{i=0}^{2n-1} (2^i a_i)$, n -bit modulus M , and LUT's input bit-width k

Result: $R = A \bmod M$

Offline:

```

1 for  $i \leftarrow 0, 1, \dots, \lceil n/k \rceil - 1$  do
2   for  $\hat{a}_i \leftarrow 0, 1, \dots, 2^k - 1$  do
3     LUT[i][ $\hat{a}_i$ ]  $\leftarrow (\hat{a}_i \times 2^{n+ki}) \bmod M$  Precompute
4   end
5 end

```

Online:

```

6 results  $\leftarrow []$ 
7 for  $i \leftarrow 0, 1, \dots, \lceil n/k \rceil - 1$  parallel do
8    $\hat{a}_i \leftarrow \sum_{j=0}^{k-1} (2^j a_{8i+j})$ 
9   results[i]  $\leftarrow$  LUT[i][ $\hat{a}_i$ ] Parallel Look up
10 end
11  $R_0 \leftarrow$  Add(results,  $\sum_{i=0}^{n-1} (2^i a_i)$ ) Adder Tree
12 results  $\leftarrow []$ 
13  $n_r \leftarrow \text{bitwidth}(R_0)$ 
14 for  $i \leftarrow 0, 1, \dots, \lceil (n_r - n)/k \rceil - 1$  parallel do
15    $\hat{r}_i \leftarrow \sum_{j=0}^{k-1} (2^j R_0[k i + j])$ 
16   results[i]  $\leftarrow$  LUT[i][ $\hat{r}_i$ ] Parallel Look up
17 end
18  $R_1 \leftarrow$  Add(results,  $\sum_{i=0}^{n-1} (2^i R_0[i])$ ) Adder (Tree)
19  $R \leftarrow R_1$  or  $R_1 - M$  or  $R_1 - 2M$  Subtract

```

where $\hat{a}_i = \sum_{j=0}^{k-1} (2^j a_{n+8i+j})$, i.e. $\hat{a}_i$ is a k -bit number. The $\frac{n}{k}$ red items can be pre-computed offline and stored using LUTs, as the modulus M is typically fixed. Specifically, we use LUTs to pre-compute and store the values of $\hat{a}_i \times 2^{n+ki} \bmod M$. A k -input and n -output LUT is used to store all the possible values of $\hat{a}_i \times 2^{n+ki} \bmod M$. For the ($n = 128$) case, an FPGA's BRAM with $36k$ -bit capacity can be configured as an 8-bit input and 128-bit output LUT and 16 BRAMs are required in total. The retrieved results from LUTs and the blue item are added together using a $\frac{n}{k} + 1$ -input adder tree to get an $(n + \log_2 d)$ -bit number, where $d = \frac{n}{k}$. The high $\log_2 d$ bits of the sum are then input to the LUTs for an additional lookup, and the result is added to the low n bits. Finally, several subtractions are performed for adjustment.

Algorithm 1 outlines the detailed process of the LUT-based modular reduction. The high n -bits are divided to d segments, denoted as $\hat{a}_i$ s, each containing k bits. Initially, each segment $\hat{a}_i$ is sent to the corresponding BRAM-based LUT to look up the pre-computed modular result of $\hat{a}_i \times 2^{n+ki} \bmod M$. Then, the d n -bit modular results, along with the low n -bit $\sum_{i=0}^{n-1} (2^i a_i)$, are added together to obtain a $n + \log_2 d$ -bit number R_0 . The high $\log_2 d$ bits of R_0 are then sent to the LUTs again, and the looked-up results are added with the low

n bits for R_0 to obtain R_1 . The second round of lookups helps refine R_1 to the final result R , assisted by several subtraction operations for adjustment.

2) *Iterative-based Method*: The iterative-based modular reduction method [13] is much more straightforward: the modular operation is performed by repeatedly subtracting the modulus from the input number until the result is smaller than the modulus. To optimize the process, shift operations are employed during each iteration to facilitate efficient subtractions. It is worth noting that because the lower n -bits of the aligned modulus M are zeros, the subtractor only needs to handle the subtraction of up to n -bit numbers.

B. FPGA Implementation for Modular Reduction

Modular reduction implementations are widely deployed on FPGAs due to their high-performance parallel computing capabilities and the flexibility they offer for designing custom solutions tailored to specific needs. These implementations are primarily divided into two categories: multiplier-based methods and LUT-based methods.

Notable examples of multiplier-based methods include Barrett's algorithm [9] and Montgomery's algorithm [10], which rely on multiple high-performance high-bit-width multipliers to achieve high throughput and low latency. However, as bit-width increases, the complexity of designing these multipliers grows significantly, making it challenging to meet throughput requirements.

On the other hand, LUT-based methods [11], [12], as discussed earlier, are better suited to support high throughput in large-number modular reduction. However, with increasing bit-width, this approach faces significant resource overhead, resulting in poor area efficiency. LUTMR [14] proposes using the FPGA's native LUTs instead of BRAMs for implementing lookup tables, aiming to reduce latency and improve LUT usage by reusing patterns. While this optimization helps at the implementation level, it does not address the inherent limitations of the method itself, resulting in only modest improvements (no more than 5% LUT savings).

III. ANALYSIS AND MOTIVATION

We conduct an in-depth analysis of the LUT-based and iterative methods for large-number modular reduction. Based on our profiling, we highlight key insights as follows:

Observation: Trade-offs between area-efficiency and latency. We analyze the area efficiency and latency of both methods across various bit-widths, as shown in Figure 2.¹ The LUT-based method offers lower latency but suffers from higher area overhead, while the iterative method exhibits better area efficiency at the cost of higher latency, especially for large bit-widths. In the LUT-based approach, lookup tables reduce online computation, thus minimizing latency. However, the area overhead increases significantly with larger bit-widths, as more BRAMs are needed to store the LUTs, and larger adder trees are required for result aggregation. Moreover, as

¹The overhead of BRAMs, adders, and subtractors is converted into LUTs, based on the utilization report from Vivado's IP generator.

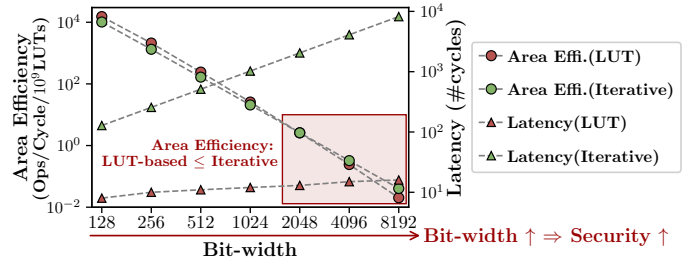


Fig. 2. Comparison of area efficiency and latency between LUT-based method and iterative-based method.

BRAM capacity is fixed, a higher bit-width limits the number of mappings a BRAM can store, increasing the number of result aggregations needed.

For example, with $n = 128$, a 36k-bit BRAM can store an 8-bit input and 128-bit output LUT, requiring 16 BRAMs and a 16-input, 128-bit adder tree. However, for $n = 2,048$, a BRAM can only store a 4-bit input and 2,048-bit output LUT, requiring 512 BRAMs and an adder tree with 512 inputs, where each adder must support at least 2,048-bit addition.

Key Insight: Achieving Pareto Optimal Design through Method Fusion. From the previous observation, the LUT-based method offers low latency but suffers from high area overhead, while the iterative method provides better area efficiency at the cost of higher latency. Therefore, it is feasible to balance area efficiency and latency by distributing the workload in a way that explores the Pareto optimal design space. Existing methods have inherent trade-offs between area efficiency and latency, with no systematic approach to achieving both high area efficiency and low latency. This gap motivates our investigation into fusing the two methods to optimize modular reduction.

By leveraging the unique advantages of each method, we aim to reduce area consumption while maintaining efficient large-number modular reduction. In this paper, we analyze hybrid workloads across both methods to achieve enhanced performance and use this analysis to design a template for balanced workload distribution. We also explore the design space, considering area overhead and latency as flexible constraints, and identify the Pareto optimal boundary of our design. Finally, we propose an automatic search method to efficiently find the optimal design that aligns with user-specified constraints.

IV. PROPOSED ALLMOD DESIGN

A. Method Fusion based on Hybrid Workloads

For formal modeling, we assume that a BRAM can store mappings from a k -bit input to an n -bit output. We use TP , the number of modular reductions per cycle, to indicate the throughput of the design. The upper bound of TP is $MaxTP = 0.5Ops/cycle$, because each modular reduction requires 2 cycles for the lookup in the BRAMs. For a $2n$ -bit input A , we divide it into two parts: the high $n - m$ bits are processed using the LUT-based method, while the low $n + m$ bits are handled by the iterative-based method.

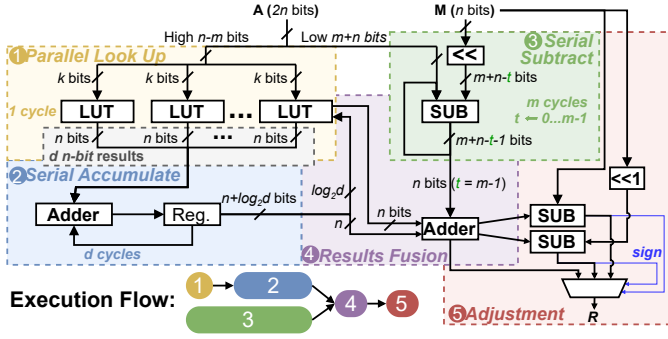


Fig. 3. ALLMod Template for balanced workload. Part ① and part ② support lookup and accumulation for LUT-based method. Part ③ supports serial subtraction for iterative-based method. Part ④ and part ⑤ are designed for fusing and adjusting the results.

Here, m is a design parameter that governs the workload distribution between the two methods. The specific derivation of this partitioning is as follows:

$$\begin{aligned}
 A &= \sum_{i=0}^{2n-1} (2^i a_i) = \sum_{i=n+m}^{2n-1} (2^i a_i) + \sum_{i=0}^{n+m-1} (2^i a_i) \\
 &= \left(\sum_{i=0}^{\frac{n-m}{k}-1} \sum_{j=0}^{k-1} (2^{n+m+ki+j} a_{n+m+ki+j}) \right) + \sum_{i=0}^{n+m-1} (2^i a_i) \\
 &= \sum_{i=0}^{\frac{n-m}{k}-1} (\hat{a}_i \times 2^{n+m+ki}) + \sum_{i=0}^{n+m-1} (2^i a_i)
 \end{aligned} \quad (2)$$

where $\hat{a}_i = \sum_{j=0}^{k-1} (2^j a_{n+m+ki+j})$. Thus, the modular reduction can also be split into two parts of workloads:

$$\underbrace{\left(\sum_{i=0}^{\frac{n-m}{k}-1} (\hat{a}_i \times 2^{n+m+ki}) \right) \bmod M}_{\text{Workload I for LUT-based method}} + \underbrace{\left(\sum_{i=0}^{n+m-1} (2^i a_i) \right) \bmod M}_{\text{Workload II for iterative method}} \quad (3)$$

In ALLMod, **workload I** is solved by the LUT-based method, and **workload II** is solved by the iterative-based method. For scalability and area efficiency considerations, we employ serial adders to implement accumulation instead of an adder tree.

B. Template Design for balanced workload

Based on hybrid workloads, we propose a template design to maximize computation efficiency. As illustrated in Figure 3, the template consists of five parts. Part ① are d BRAMs for storing the pre-computed results of $\hat{a}_i \times 2^{n+m+ki} \bmod M$, where $d = \frac{n-m}{k}$. The high $n-m$ bits are fed into the BRAMs for parallel lookup. Part ② uses an adder for serially accumulating the d results from the BRAMs. Part ③ consists of a subtractor and a shifter for iterative subtractions. Part ④ is an adder for fusing the results from the two methods, including the low n bits of the accumulation results from part ②, the second round lookup result from part ①, and the serial subtraction result from part ③. Part ⑤ deploys several subtractors and a multiplexer for adjusting the final result.

The dataflow is also illustrated in Figure 3. For the **left workload**, the high $n-m$ bits are sent to part ① for parallel lookup, which takes one cycle. The results are then serially accumulated in part ② over d cycles. After accumulating the $n+\log_2 d$ -bit result in part ②, the high $\log_2 d$ bits are sent back to part ① for a second round of lookup. Meanwhile, for the **right workload**, the low $n+m$ bits are sent to part ③, where they undergo $m-1$ cycles of subtraction. In part ④, the results from the previous three parts—part ①, part ②, and part ③—are added together for fusion. The final result is then adjusted using several subtractions in part ⑤. Regarding resource reuse, since the execution of part ② and part ③ does not overlap with that of part ④ and part ⑤, resource reuse is possible. The adder in part ② can be reused in part ④ for aggregation, and the subtractor in part ③ can also be reused in part ⑤ for adjustment. This further optimizes the design by reducing redundant hardware resources and enhancing efficiency.

Given that the two workloads are executed in parallel, the latencies of the two workloads should ideally be equal to ensure a balanced workload and minimize resource waste. The latency of the left workload is $d+1$ cycles ($d = \frac{n-m}{k}$), and the latency of the right workload is m cycles. Therefore, the following equation should be satisfied:

$$\frac{n-m}{k} + 1 = m \Rightarrow m = \frac{n+k}{k+1} \quad (4)$$

Apart from the BRAMs, the adder and subtractor in the template need to be copied $d \times TP$ times to ensure the throughput.

The template can reduce the area overhead of both BRAMs and adders/subtractors. Compared to the LUT-based method, the template can achieve $\frac{m}{n} = \frac{n+k}{n(k+1)} \approx \frac{1}{k+1}$ BRAM savings. Moreover, the original LUT-based method needs $\frac{2n}{k}$ adders/subtractors, while ours just needs $d = \frac{n-m}{k} =$ pieces to reach maximum throughput $MaxTP$. For $n = 8,192$, a BRAM can only store mappings with 2-bit input, meaning $k = 2$. Consequently, our template can save up to 50% of the required BRAMs and over 65% of the required adders/subtractors.

C. Template-aided Design Space Exploration

To accommodate a broader range of design requirements, we explore the design space of ALLMod. Using the proposed template, which focuses on balanced workloads, we investigate the design space by considering both latency and area constraints as key parameters.

1) *Parallel-Serial Hybrid for More Strict Latency Constraint:* In cases where stricter latency requirements are imposed, the single-pass algorithm must be completed in fewer cycles. Since the iterative method cannot be parallelized, we propose reducing its workload while shifting more work to the table lookup method. Additionally, we aim to enhance the parallel processing capacity of specific components within the table lookup method.

To achieve this, we introduce a small adder tree into part ② of the template, which works alongside the serial accumulator to accelerate the accumulation process. This setup allows the

workload in the left part of the design to be completed in fewer cycles, helping to meet the stricter latency constraints. When configured with an adder tree that has x inputs, part ② can be completed in $\frac{n-m}{k} - x$ cycles, while part ③ will still take m cycles. The optimal design point is determined by finding m and x that satisfy the following conditions:

$$\max(1 + \frac{n-m}{k} - x, m) \leq \text{Latency}_{req} \quad (5)$$

where Latency_{req} is the user-specified number of cycles required for the design.

2) *More Strict Area Constraint*: When area constraints become more stringent, we assign a larger portion of the workload to part ③ (iterative subtraction) to reduce the BRAM overhead of part ② (parallel lookup). However, since the workload in part ③ cannot be accelerated in parallel, we need to ensure throughput by duplicating more of the components in part ③.

Assuming that part ③ is duplicated y times, its throughput is approximately $\frac{y}{m}$ Ops/cycle. To guarantee throughput, we set $y = m \times TP$. Similarly, part ② must be duplicated $d \times TP$ times. Consequently, the task is to determine the value of m such that the design meets both area and throughput requirements:

$$\begin{aligned} \frac{n-m}{k} \times \text{Area}_{BRAM} + d \times TP \times \text{Area}_{Adder} \\ + m \times TP \times \text{Area}_{Subtractor} \leq \text{Area}_{req} \end{aligned} \quad (6)$$

where Area_{req} is the user-specified area limit, and $\text{Area}_{BRAM}/\text{Adder}/\text{Subtractor}$ represents the area of a BRAM/adder/subtractor.

3) *Automatic Search*: The analysis of the design space only focuses on either lower latency or smaller area consumption. In practice, the design usually has constraints on both latency and area. Therefore, based on the previous analysis, we propose an automatic search approach to find the Pareto optimal design schemes that meet various constraints provided by users. As shown in Algorithm 2, our search method takes the bit-width n , and user-specified constraints as input, and outputs a list of Pareto optimal schemes. Users' constraints include latency requirement Latency_{req} , area requirement Area_{req} , and expected throughput TP .

First, we enumerate m from 0 to n to explore different workload segmentations. Specifically, $m = 0$ corresponds to the pure LUT-based method, while $m = n$ corresponds to the pure iterative method. Next, we enumerate the input width of the adder tree, Width_{Tree} , from 0 to $n - m$. When $\text{Width}_{Tree} = 0$, the template with a balanced workload, as introduced in Section IV-B, is used; when $\text{Width}_{Tree} = n - m$, part ② is entirely implemented using an adder tree for parallel accumulation. Each pair of values (m, Width_{Tree}) forms a key *parameter pair* for a candidate scheme.

For each *parameter pair*, we compute the cycles required to complete the modular reduction, as outlined in Section IV-B, and verify whether the latency requirement is met. Then, we calculate the number of BRAMs, adders, and subtractors needed for the scheme and estimate the corresponding area

Algorithm 2: Automatic Design Space Search.

Data: Bit-width n , Input bit-width of BRAM(LUT) k , user-specified latency and area constraints Latency_{req} and Area_{req} , expected throughput TP

Result: Pareto optimal scheme list OptSchemeList

```

1  $\text{SchemeList} \leftarrow []$ 
2 for  $m \leftarrow 0 \dots n$  do
3   for  $\text{Width}_{Tree} \leftarrow 0 \dots n - m$  do
4      $\text{Latency}_{LUT} \leftarrow 1 + \max(\frac{n-m}{k} - \text{Width}_{Tree}, \log_2(\text{Width}_{Tree}))$ 
5      $\text{Latency}_{iter} \leftarrow m$ 
6      $\text{Latency} \leftarrow \max(\text{Latency}_{LUT}, \text{Latency}_{iter})$ 
7     if  $\text{Latency} \leq \text{Latency}_{req}$  then
8        $N_{BRAMs} \leftarrow \frac{n-m}{k}$ 
9        $N_{Adders_{Tree}} \leftarrow 2 \times \text{Width}_{Tree} - 1$ 
10       $N_{Adders_{Serial}} \leftarrow (\frac{n-m}{k} - \text{Width}_{Tree}) \times TP$ 
11       $N_{Adders} \leftarrow N_{Adders_{Tree}} + N_{Adders_{Serial}}$ 
12       $N_{Subtractors} \leftarrow m \times TP$ 
13       $\text{Area} \leftarrow \text{AreaEstimate}(N_{BRAMs}, N_{Adders}, N_{Subtractors})$ 
14      if  $\text{Area} \leq \text{Area}_{req}$  then
15         $\text{Scheme}_{feasible} \leftarrow (m, \text{Width}_{Tree}, \text{Cycles}, \text{Area})$ 
16         $\text{SchemeList.append}(\text{Scheme}_{feasible})$ 
17      end
18    end
19  end
20 end
21  $\text{OptSchemeList} \leftarrow \text{FindParetoOptimal}(\text{SchemeList})$ 

```

overhead. Feasible schemes that satisfy both latency and area constraints are added to a candidate list, SchemeList . Finally, we identify the Pareto optimal schemes from this list. Through such an automatic search process, users can easily identify the Pareto optimal design schemes that meet their specific constraints.

In addition, our approach efficiently identifies Pareto optimal schemes. By enumerating both m and Width_{Tree} , the total number of feasible schemes, denoted as S , can be at most n^2 . Using a priority queue to sort and select the optimal schemes, the time complexity for finding the Pareto optimal solutions is $O(S \log S) = O(n^2 \log n)$. Even for very large values of n , such as 8,192, the entire search process completes in approximately ten minutes².

V. EVALUATIONS

A. Evaluation Methodology

We compare ALLMod with the standard LUT-based method [12] and iterative method [13]. We implement a 128-bit prototype system for ALLMod on FPGA using Verilog, with a working frequency set to 200MHz. To validate the

²On a laptop with Intel i9-12900H CPU and 32GB memory.

TABLE I
COMPARISON OF LUT-BASED METHOD, ITERATIVE METHOD, AND ALLMod (TEMPLATE), ALL ALIGNED TO $MaxTP$.

Bit-Width (n)	Area Efficiency ($Ops/Cycles/10^9 LUTs$)			Latency (Cycles)			Breakdown ($BRAMs; Adders; Subtractors$)			Hybrid Workloads
	LUT-based [12]	Iterative [13]	ALLMod (Improve)	LUT-based	Iterative	ALLMod	LUT-based	Iterative	ALLMod	ALLMod
128	15258.79	10172.53	25040.06 ($1.65\times$)	9	128	20	16;3;1	0;0;64	15;8;8	113;143
256	2111.49	1326.85	4521.12 ($2.14\times$)	11	256	37	37;73;1	0;0;128	32;16;16	224;288
512	241.60	165.86	550.18 ($2.28\times$)	12	512	78	86;171;1	0;0;256	73;37;37	438;586
1024	25.75	20.73	61.05 ($2.37\times$)	13	1024	176	205;409;1	0;0;512	171;86;86	853;1195
2048	2.59	2.59	6.45 ($2.49\times$)	14	2048	415	512;1024;1	0;0;1024	410;205;205	1638;2458
4096	0.24	0.32	0.65 ($2.71\times$)	16	4096	1029	1366;2731;1	0;0;2048	1024;512;512	3072;5120
8192	0.02	0.04	0.06 ($3.00\times$)	17	8192	2736	4096;8191;1	0;0;4096	2731;1366;1366	5461;10923

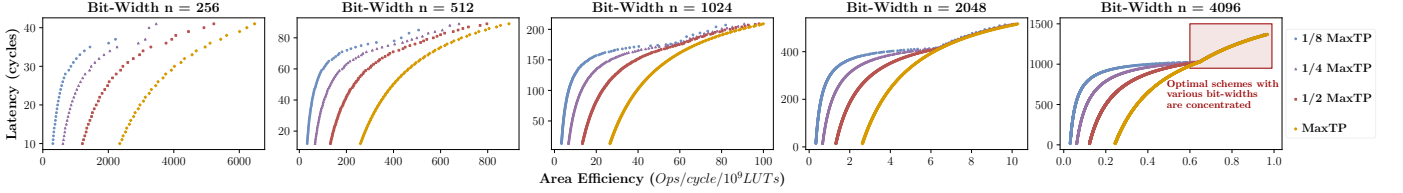


Fig. 4. Pareto optimal schemes of ALLMod for various throughputs and bit-widths. The optimal schemes with high area efficiency gradually converge as the bit-width increases.

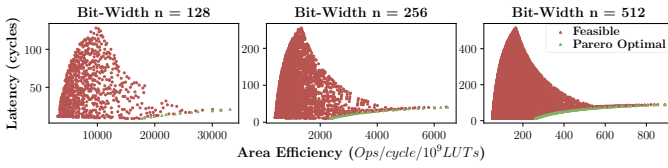


Fig. 5. Visualization of searched schemes for various bit-width at maximum throughput $MaxTP$. Red points indicate the feasible schemes and green points indicate the Pareto optimal schemes.

timing and functionality of our models, we synthesized them using the Xilinx Vivado Design Suite [15]. For evaluation, we use $\frac{Ops/cycle}{10^9 LUTs}$ as the metric of area efficiency. Given that the working frequency for both the baseline and our method is primarily determined by the latency of the n -bit adder/subtractor, we utilize *cycles* as a standardized metric for latency comparison, assuming a similar frequency across implementations. This allows for a fair comparison of execution time independent of absolute clock speeds. The overhead of BRAMs, adders, and subtractors is converted with LUTs, based on the utilization report from the IP generator of Vivado.

B. Overall Performance

Table I compares the performance of our ALLMod template with the baseline method. The methods are all aligned to the maximum throughput $MaxTP$ that the LUT-based method can achieve. We show the area efficiency, latency, and the breakdown of resources for each method, and segmentation for hybrid workloads of ALLMod.

For small bit-width $n = 128$, ALLMod achieves the area efficiency of 25,000 $Ops/Cycles/10^9 LUTs$, which is $1.65\times$ higher than the LUT-based method. For large number modular reduction with $n = 8,192$, ALLMod achieves up to $3.0\times$ area efficiency improvement over the LUT-based method. It's worth noting that although the LUT-based method has lower latency, it needs too much area overhead to achieve the same throughput as ALLMod.

The breakdown results demonstrate that ALLMod can save a significant number of BRAMs and adders/subtractors. For $n = 8,192$, ALLMod saves over 30% of BRAMs and 66%

of adders/subtractors compared to the LUT-based method. As the bit-width increases, the proportion of workload allocated to the right parts in the template gradually increases, which illustrates the significance of introducing a hybrid workload design for optimizing the LUT-based method.

C. Pareto Optimal with various throughput requirements

Figure 4 shows the Pareto optimal schemes of ALLMod for various throughputs and bit-widths. We iterate the throughput requirement from $MaxTP/8$ to $MaxTP$ for each bit-width. As the bit-width increases, the Pareto optimal schemes for different throughputs are more concentrated in the region with high area efficiency, meaning that the design gradually converges. Such a convergence demonstrates that ALLMod can effectively explore the design space and find optimal schemes for guiding the design.

D. Design space exploration with automatic search

Figure 5 visualizes the design space exploration results of ALLMod for various bit-widths. The distributions of feasible schemes under different bit-widths are basically the same. A significant concentration of feasible schemes is observed within the region of lower area efficiency and high latency, whereas a small number of feasible solutions are identified in the region of higher area efficiency. This distribution underscores the criticality of identifying Pareto optimal schemes. Our automatic search effectively filters out schemes with low area efficiency and high latency and finds area-efficient schemes with low latency.

VI. CONCLUSION

This paper presents ALLMod, an area-efficient method for large-number modular reduction, combining LUT-based and iterative approaches. We then design a balanced workload template to guide the segmentation. Additionally, we present an automatic search approach to explore the design space of ALLMod based on this template. Evaluations show that ALLMod outperforms the LUT-based method in terms of area efficiency and latency.

REFERENCES

- [1] A. Acar, H. Aksu, A. S. Uluagac, and M. Conti, "A survey on homomorphic encryption schemes: Theory and implementation," *ACM Computing Surveys (Csur)*, vol. 51, no. 4, pp. 1–35, 2018.
- [2] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," in *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*, 2019, pp. 203–225.
- [3] U. Fiege, A. Fiat, and A. Shamir, "Zero knowledge proofs of identity," in *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, 1987, pp. 210–217.
- [4] N. Koblitz, "Elliptic curve cryptosystems," *Mathematics of computation*, vol. 48, no. 177, pp. 203–209, 1987.
- [5] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [6] L. Chen, D. Moody, A. Regenscheid, and A. Robinson, "Digital signature standard (dss)," 2023.
- [7] J. Bos, M. Kaihara, T. Kleinjung, A. K. Lenstra, and P. L. Montgomery, "On the security of 1024-bit rsa and 160-bit elliptic curve cryptography," 2009.
- [8] B. Schneier, *Applied cryptography: protocols, algorithms, and source code in C*. john wiley & sons, 2007.
- [9] P. Barrett, "Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor," in *Conference on the Theory and Application of Cryptographic Techniques*. Springer, 1986, pp. 311–323.
- [10] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [11] B. Devlin, "Blockchain acceleration using fpgas—elliptic curves, zk-snarks, and vdfs," *ZCASH Foundation*, 2019.
- [12] E. Öztürk, "Design and implementation of a low-latency modular multiplication algorithm," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 6, pp. 1902–1911, 2020.
- [13] W. A. S. Wijesinghe, "A high-speed hardware algorithm for modulus operation and its application in prime number calculation," 2024. [Online]. Available: <https://arxiv.org/abs/2407.12541>
- [14] A. Opatanian and M. Ikeda, "Lookup table modular reduction: A low-latency modular reduction for fast ecc processor," in *2023 IEEE Symposium in Low-Power and High-Speed Chips (COOL CHIPS)*. IEEE, 2023, pp. 1–6.
- [15] T. Feist, "Vivado design suite," *White Paper*, vol. 5, 2012.